

DEVELOPMENT OF AUTONOMOUS ROBOT CAR: SS2026 ELE SYSTEM ENGINEERING

Authors: Deemana Dewmini Mirihella Adikaramalage; Shehan Nihindu Thenuka Liyanage Liyanage; Shamal Chathura Gamalath Kankanamalage; Ishan Wickramathunga

Date: 26/07/2026

ABSTRACT

An autonomous vehicle (AV) is a vehicle capable of operating without direct human intervention. Autonomous driving has become one of the most rapidly advancing fields in modern transportation, with significant research and development efforts focused on improving vehicle intelligence, safety, and reliability. Although fully autonomous vehicles remain an active area of research, several advanced driver assistance systems (ADAS), such as Adaptive Cruise Control (ACC), are already commercially available. These systems can partially automate driving tasks while still requiring the driver to remain attentive and intervene when necessary. As part of the System Engineering course, the objective of this project was to design and develop a prototype autonomous robot car using a structured systems engineering approach. The prototype was required to follow a predefined path, detect and avoid static obstacles, and resume line following after obstacle avoidance. The development process began with requirements engineering and SysML-based system modelling, followed by mechanical design, hardware integration, and software implementation.

A PID-based line-following controller was employed to improve tracking accuracy and ensure smooth navigation, while ultrasonic sensors were integrated for obstacle detection and avoidance. Experimental evaluation demonstrated reliable line-following performance and an obstacle avoidance success rate of approximately 90%. The developed prototype provides an effective, low-cost autonomous navigation platform and demonstrates the practical application of systems engineering principles in embedded autonomous robotic systems.

CHAPTER 1: INTRODUCTION

According to the Society of Automotive Engineers (SAE), driving automation is classified into six levels, ranging from Level 0 (no driving automation) to Level 5 (full driving automation) [1]. Automotive manufacturers, universities, and research institutions continue to invest significant effort in advancing autonomous driving technologies to achieve higher levels of automation.

The objective of this project was to develop a prototype autonomous robot car that demonstrates the fundamental capabilities of an autonomous navigation system. Although the prototype operates in a controlled environment and is not directly comparable to a road vehicle, its functionality reflects several key characteristics associated with higher levels of driving automation. The project focused on achieving the following objectives:

Primary Objectives

1. Develop a robot capable of accurately following a predefined line.
2. Enable the robot to detect and avoid a static obstacle, reacquire the line, and continue autonomous navigation.

Secondary Objectives

1. Implement a 180° turning manoeuvre upon encountering a second obstacle and continue line following in the reverse direction.
2. Improve the robot's travelling speed while maintaining stable operation.
3. Enhance the smoothness and accuracy of line-following performance through controller optimization.

CHAPTER 2: SYSTEM ENGINEERING APPROACH

Prior to implementation, a systems engineering approach was adopted to establish a structured development process. Requirements engineering was first carried out to identify the functional and non-functional requirements of the autonomous robot. SysML was then used to document the system architecture and behaviour through Requirement, Block Definition, Activity, Sequence, Use Case, and Package diagrams. These models provided a systematic representation of the system throughout the development process and are available in the project repository [2].

The project development followed a **Multiple V Model** for embedded systems, as shown in Figure 1. Instead of completing the entire system within a single V-cycle, the project was divided into several incremental development stages, each represented by its own V-model. The first V-cycle focused on the mechanical design, 3D modelling, assembly, and hardware validation. Subsequent V-cycles addressed the implementation and testing of the line-following controller and the obstacle avoidance functionality. Each stage was individually designed, implemented, verified, and validated before proceeding to the next, enabling incremental system integration while reducing development risk.

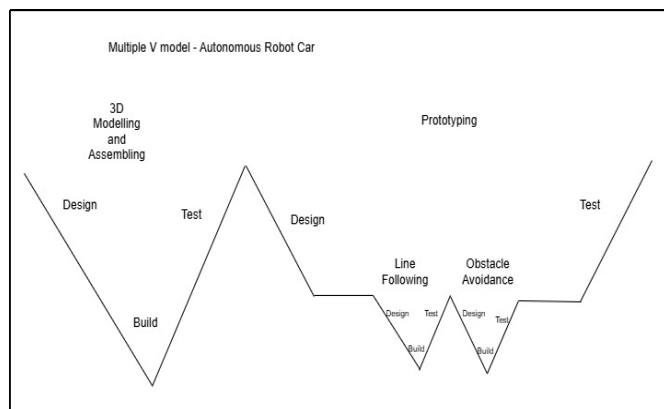


Figure 1: Multiple V Model adopted for the development of the autonomous robot car

CHAPTER 3: HARDWARE AND SOFTWARE IMPLEMENTATION

3.1 Hardware Implementation

To develop the prototype, the following hardware components were provided and utilized. The list of components and their required quantities are shown in the table below.

Part	No of items
IR sensor (KY-033)	2
Ultrasonic sensor (HC-SR04)	2
Motors	2

Side Wheels	2
Arduino Uno	1
L298N Dual H-Bridge Motor Driver	1
Free wheel	1
Battery (11.1V / 3200mAh)	1
Breadboard	1
Switch	1

Table 1: Components Used

The project required the design and fabrication of custom holders for the IR sensors, ultrasonic sensors, motors, battery, breadboard and Arduino Uno. The components were designed using **SolidWorks**, while **Bambu Studio** was used to verify that the estimated 3D printing time for each part remained within the specified limit of time. Particular attention was given to the IR sensor holder, which was designed to securely mount the sensors while maintaining sufficient ground clearance to prevent contact with the track. In addition, an adjustable sliding mechanism was incorporated to allow lateral positioning of the IR sensors, enabling precise alignment with the line and improving tracking performance.

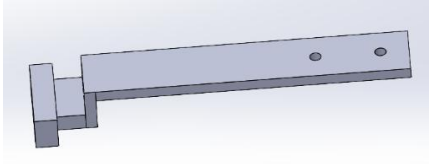


Figure 2: IR Sensor Holder

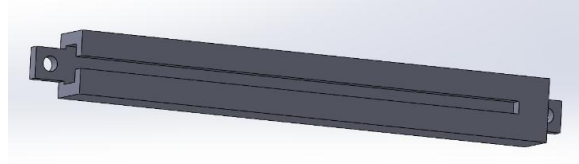


Figure 3: IR Sensor Sliding Bar

After fabrication, the prototype was assembled, and the electronic components were installed and interconnected according to the system block diagram [3].

3.2 Software Implementation

The software was developed incrementally in two phases, with continuous optimization carried out throughout the implementation. The first phase focused on autonomous line following using two infrared (IR) sensors. The IR sensors produce a binary output, where **1** represents the detection of the black line and **0** represents the white background. Based on these sensor readings, the robot determines its position relative to the line and adjusts its steering accordingly.

IR Left Sensor	IR Right Sensor	Robot Action
1	1	Move straight forward
1	0	Adjust direction by drifting left
0	1	Adjust direction by drifting right
0	0	Turn toward the last detected line direction

Table 2: Robot Action for IR Sensor Values

To achieve smooth and stable navigation, a Proportional–Integral–Derivative (PID) controller was implemented. The controller continuously calculates the tracking error from the IR sensor inputs and adjusts the left and right motor speeds to minimize this error. The proportional term provides immediate correction while the derivative term reduces oscillations by anticipating changes in the tracking error.

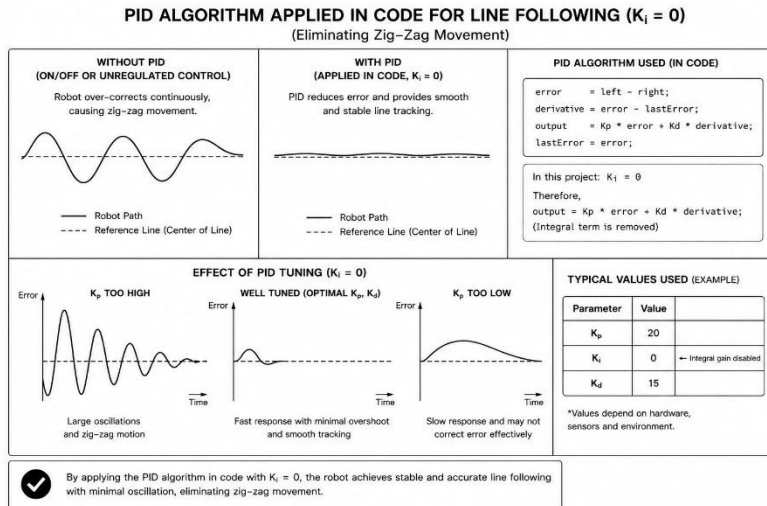


Figure 4: PID Controller in Autonomous Robot Car

predefined manoeuvring sequence consisting of six stages: **Stop, Turn away from the obstacle, Turn back to align parallel with the original path, Move alongside the obstacle, Turn towards the line, and Reacquire the line.**

Following successful implementation of the obstacle avoidance strategy, further software optimizations were introduced. The motor speed was increased during straight-line motion to improve travelling speed while maintaining stable tracking. In addition, the robot was programmed to perform a 180° turn when a second obstacle was encountered, allowing it to reverse direction and continue autonomous navigation. The manoeuvring sequence was also refined to produce smoother turns and improve the overall reliability of obstacle avoidance.

CHAPTER 4: SIMULATION AND TESTING RESULT

Before assembling the physical prototype, the circuit was simulated using TinkerCAD to verify the hardware configuration and basic software functionality. Although TinkerCAD does not support simulation of the complete autonomous navigation system, it provided an effective platform for validating the circuit design, verifying component connections, and identifying implementation errors prior to hardware integration.

Following assembly, the prototype was experimentally evaluated in two development stages: **line following** and **obstacle avoidance**. During the line-following stage, multiple iterations of PID controller tuning were performed to achieve stable and smooth navigation. After optimization, the robot was able to accurately follow both straight and curved sections of the track with minimal oscillation. The obstacle avoidance stage involved repeated testing and refinement of the manoeuvring sequence to improve the reliability of obstacle detection, avoidance, and line reacquisition. Through successive optimization, the prototype achieved an obstacle avoidance success rate of approximately **90%** under the test conditions.

Test	Result
Line following success	100%
Obstacle avoidance success	90%
Second obstacle U turn	100%
Average Obstacle Detection Distance	36 cm
Maximum Operating Speed	120 PWM

Table 3: Testing Results of the Autonomous Robot Car

In addition to obstacle manoeuvring, the 180° turn required for the second obstacle was optimized and validated through multiple test iterations.

Chapter 5: CHALLENGES ENCOUNTERED AND SOLUTIONS

From the initial assembly stage to the final testing phase, we encountered various challenges throughout the development process. However, through continuous troubleshooting, optimization, and testing, we were able to overcome these challenges and successfully complete the project. The main challenges encountered and the solutions implemented are summarized in the table below.

Challenge	Solution
The component designed to keep the chassis parallel to the floor (compensating for the height difference between the side wheels and the free wheel) had slight dimensional differences compared to the chassis mounting hole.	The component was trimmed and reshaped to achieve a proper fit within the chassis hole.
Some component holders were too tight, preventing the components from being inserted properly.	The holders were filed and adjusted to provide sufficient clearance for component installation.
The robot was unable to follow a straight line accurately due to physical differences between the wheels.	Individual speed values were assigned to each wheel and calibrated until the robot achieved straight-line movement.
The robot followed the line with noticeable oscillation.	The initial simple line-following algorithm was replaced with a PID controller to achieve smoother and more stable movement.
When the robot drifted away from the black line and both sensors detected white, it was unable to recover and locate the line again.	A memory-based approach was implemented using a variable called last direction , allowing the robot to turn gradually toward the last detected direction of the black line and recover its path.
After implementing obstacle avoidance, the robot stopped following the line and unexpectedly turned left or right.	Initially, a basic ultrasonic detection routine was added that continuously ran until the end of the sensor logic. This blocking approach prevented the IR sensor logic from executing properly. The ultrasonic code was redesigned using a non-blocking interleaved approach, allowing obstacle detection and line following to operate simultaneously.
The robot started executing the obstacle manoeuvring logic even when no obstacle was present due to occasional incorrect sensor readings.	Instead of relying on a single ultrasonic reading, obstacle detection was confirmed through four consecutive measurements before initiating the manoeuvre.

Table 4: Challenges Encountered and Solutions

CHAPTER 6: CONCLUSION AND FUTURE IMPROVEMENTS

Autonomous vehicles (AVs) are a rapidly developing field that continues to receive significant research interest and investment. This project provided valuable practical experience in the development of an autonomous robotic vehicle and demonstrated the challenges involved in achieving reliable autonomous operation. Through this project, it was observed that successful autonomous navigation requires not only accurate hardware integration but also continuous software optimization and testing. The final prototype was able to demonstrate key autonomous functionalities, including line following and obstacle manoeuvring. However, the testing process also highlighted the complexity of achieving consistent performance in real-world conditions, where mechanical variations, sensor limitations, and environmental factors can significantly affect the system behaviour.

Although the project objectives were achieved, there are still areas that require further improvement to enhance the reliability, smoothness, and accuracy of the robot. Future developments could focus on the following improvements:

- Further tuning and optimization of the PID controller to achieve smoother and more stable line-following performance.
- Improving the obstacle manoeuvring algorithm to increase reliability and ensure successful execution, particularly during curved paths and complex scenarios.
- Integrating an additional IR sensor to improve line detection accuracy and allow more precise navigation.
- Enhancing the ultrasonic sensor algorithm to minimize false readings and prevent unintended obstacle detection caused by sensor noise or glitches.

In conclusion, this project demonstrated the importance of iterative testing, optimization, and integration between mechanical design and software development in autonomous systems. While the prototype achieved the intended functionality, further refinement and research would enable the development of a more robust and efficient autonomous vehicle capable of handling a wider range of operating conditions.

REFERENCES

- [1] S. International, "J3016_202104 - Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles," 30/04/2021. [Online]. Available: https://www.sae.org/standards/content/j3016_202104/ . [Accessed 14/07/2026].
- [2] Github, "Prototyping-E2-E5 Diagrams," 07/2026. [Online]. Available: <https://github.com/DeemanaDewminiMirihehlla/Prototyping-E2-E5/tree/main/Diagrams> .
- [3] Github, "Prototyping-E2-E5 Diagrams/Block Diagram," 07/2026. [Online]. Available: <https://github.com/DeemanaDewminiMirihehlla/Prototyping-E2-E5/blob/main/Diagrams/Block%20Diagram.pdf> .

TASK ALLOCATION TABLE

Task	Short Summary	Shamal Chathura		Ishan Wickramathunga		Shehan Nihindu		Deemana Dewmini	
		Todo (Deadline)	Done (Finished Date)	Todo (Deadline)	Done (Finished Date)	Todo (Deadline)	Done (Finished Date)	Todo (Deadline)	Done (Finished Date)
1 Task 1	First System Engineering Modelling with SysML Diagrams	Use Case Diagram (24/05/2026)	24/05/2026	Class Diagram (24/05/2026)	23/05/2026	Sequence Diagram (24/05/2026)	24/05/2026	Block Diagram and Requirement Diagram (24/05/2026)	24/05/2026
2 Task 2	TinkerCAD Simulation and System specification with SysML	Activity Diagram(24/05/2026)	24/05/2026	TinkerCAD Simulation (24/05/2026)	24/05/2026	Package Diagram24/05/2026	24/05/2026	Block Definition Diagram & Internal Block Diagram 24/05/2026	24/05/2026
3 Task 3	3D Designing of the Parts	IR Sensor Holder & Sliding Bar (06/05/2026)	06/05/2026	Ultrasonic Sensor and Arduino Uno Holders (06/05/2026)	06/05/2026	Breadboard and Battery Holders (06/05/2026)	06/05/2026	Motor Holders and the component designed to keep the chassis parallel to the floor (06/05/2026)	06/05/2026
4 Task 4	Hardware Assembly & Wiring	Mount parts to the chassis	24/05/2026	Establish electrical connections	24/05/2026	Mount parts to the chassis	24/05/2026	Establish electrical connections	24/05/2026
5 Task 5	Implementing and improving the Code and conducting prototype troubleshooting	Prototype troubleshooting	03/07/2026	Code Implementation and Improvement	03/07/2026	Prototype troubleshooting	03/07/2026	Code Implementation and Improvement	03/07/2026
6 Task 6	Final Documentation (Presentation and Final Report)	Presentation Preparation (05/07/2026)	04/07/2026	Presentation Preparation (05/07/2026)	04/07/2026	Final Report Preparation (26/07/2026)	20/07/2026	Final Report Preparation (26/07/2026)	20/07/2026